

Product Requirements Document

Tango - YouTube to Anki Vocabulary Pipeline

Version 0.4.0 | July 2026

1. Overview

Tango is a command-line pipeline that converts YouTube video transcripts into Anki flashcard packages. A user provides a video ID and a target deck name. The system extracts vocabulary, checks for existing cards, fetches definitions and example sentences, and produces an importable .apkg file. The tool is designed for language learners who want to build vocabulary from authentic video content without manually creating cards.

1.1 Problem Statement

Language learners frequently encounter new words in YouTube content but have no efficient way to convert those encounters into structured review material. Manual card creation is time-consuming and inconsistent. Existing tools either require browser plugins, have no duplicate detection, or provide definitions only in English regardless of the target language.

1.2 Goals

The system must extract meaningful vocabulary from any YouTube video with available captions. It must avoid creating duplicate cards for words the user already knows. It must support at least 40 languages for subtitle extraction and return example sentences in the original language of the video. It must be runnable by a technically literate user with minimal setup.

1.3 Non-Goals for Current Version

The system does not provide a graphical interface. It does not support audio on cards. It does not automatically sync with AnkiWeb. It does not recommend videos. It does not support platforms other than YouTube.

2. Users

The primary user for v0.4.0 is a technically literate language learner comfortable with a terminal, a .env file, and basic Anki usage. They are learning a language by watching authentic content and want to systematically build vocabulary from that content. They may be learning French, Spanish, German, Japanese, or any of the 40 supported languages.

Secondary users are developers who want to build on the pipeline or integrate it into their own tools. The codebase is structured to support this with clear module boundaries and a SQLite state layer.

3. User Stories

3.1 As a French learner, I want to run the pipeline on a French YouTube video and get Anki cards with French example sentences so that I learn words in context rather than in isolation.

3.2 As a user with an existing Anki deck, I want the pipeline to skip words I already have cards for so that I do not create duplicate content.

3.3 As a user studying French with English as my first language, I want to see the English definition of a French word alongside French example sentences so that I understand the meaning without losing the original context.

3.4 As a user, I want to be prompted when a word might already exist in my deck under a slightly different form so that I can decide whether to create a new card or skip it.

3.5 As a user, I want cards that still appear when no dictionary definition is found, using a sentence from the video as the example, so that I do not lose rare or informal vocabulary.

4. Functional Requirements

4.1 Transcript Extraction

The system must accept a YouTube video ID or URL and return a clean transcript string. It must prefer manually created transcripts over auto-generated ones. It must support proxy configuration for IP-blocked environments. It must handle all error conditions including age-restricted videos, disabled captions, and IP blocks with clear error messages.

4.2 Language Resolution

The system must resolve the target language from either an explicit flag or the deck name. It must support 40 languages with name variants in English, French, Spanish, German, and each language's own name. It must warn the user clearly when no language can be inferred and must not silently default to English.

4.3 NLP Processing

The system must tokenize the transcript using spaCy and return lemmatized vocabulary filtered by part of speech. It must preserve first-appearance ordering. It must count word frequency for future ML use. It must not filter words by length or exclude stop words, as both categories are valuable for beginners.

4.4 Deck Checking

The system must query the user's Anki deck for existing card fronts via AnkiConnect. It must classify each incoming lemma as skip, queue for review, or new using a three-condition fuzzy match. It must detect sentence-structured decks and disable fuzzy matching for them. It must store words to a backlog when Anki is unavailable and process

them when the user requests.

4.5 Definition Fetching

The system must fetch example sentences and synonyms in the original transcript language. It must fetch definitions in the user's chosen output language. It must try Merriam-Webster first for English definitions and fall back to dictionaryapi.dev. It must cache all successful lookups to avoid redundant API calls. All text fields must be capped at 256 characters.

4.6 Card Generation

The system must produce a valid .apkg file importable by Anki. Cards must contain the word, grammatical class, definition, up to two dictionary example sentences, a transcript example sentence, synonyms, and antonyms. Cards for words with no definition found must show the transcript example sentence. The card template must render correctly in both Anki light and dark modes.

5. Non-Functional Requirements

5.1 Performance

Processing time is currently dominated by sequential API calls. A 3-minute video with 100 unique words takes approximately 12 minutes. This is acceptable for v0.4.0. Async I/O is targeted for v1.0.0 and expected to reduce processing to under 3 minutes for the same input.

5.2 Reliability

The pipeline must not crash on a single word failure. API errors must be caught, logged, and treated as not-found cases. The SQLite cache must prevent re-fetching words already processed across multiple runs.

5.3 Testability

All modules must have unit tests that do not require network access, a running Anki instance, or installed spaCy models. Integration tests may require these but must be gated behind a marker. The test suite must pass in under 3 minutes excluding integration tests.